

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Обучение классификаторов средствами библиотеки PyTorch»

Выполнил:

Студент 4 курса

Группы ИИ-26

Кушнеревич Е.А

Проверила:

Андренко К.В.

Цель работы: Научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения

Ход работы:

1. Выполнить конструирование своей модели СНС, обучить ее на выборке по заданию (использовать **torchvision.datasets**). Предпочтение отдавать как можно более простым архитектурам, базирующимся на базовых типах слоев (сверточный, полносвязный, подвыборочный, слой нелинейного преобразования). Оценить эффективность обучения на тестовой выборке, построить график изменения ошибки (matplotlib);
2. Ознакомьтесь с state-of-the-art результатами для предлагаемых выборок (из материалов в сети Интернет). Сделать выводы о результатах обучения СНС из п. 1;
3. Реализовать визуализацию работы СНС из пункта 1 (выбор и подачу на архитектуру произвольного изображения с выводом результата);
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

№ варианта	Выборка	Размер исходного изображения	Оптимизатор
8	CIFAR-10	32X32	Adam

Код программы:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader

import torchvision
import torchvision.transforms as transforms

import numpy as np
import matplotlib.pyplot as plt
import random
from PIL import Image
from collections import defaultdict

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Используемое устройство:", device)

CLASSES = ('airplane', 'automobile', 'bird', 'cat', 'deer',
            'dog', 'frog', 'horse', 'ship', 'truck')
```

```

MEAN = (0.4914, 0.4822, 0.4465)
STD = (0.2470, 0.2435, 0.2616)

transform_train = transforms.Compose([
    transforms.RandomCrop(32, padding=4),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

transform_test = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(MEAN, STD),
])

train_set = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True, transform=transform_train)
test_set = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True, transform=transform_test)

BATCH_SIZE = 128
train_loader = DataLoader(train_set, batch_size=BATCH_SIZE, shuffle=True, num_workers=2)
test_loader = DataLoader(test_set, batch_size=256, shuffle=False, num_workers=2)

print(f"Train:    {len(train_set)},    Test:    {len(test_set)},    размер    изображения: {train_set[0][0].shape}")

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(128 * 4 * 4, 256)
        self.fc2 = nn.Linear(256, num_classes)
        self.dropout = nn.Dropout(0.4)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

model = SimpleCNN().to(device)
n_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(model)
print(f"Обучаемых параметров: {n_params:,}")

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)
N_EPOCHS = 10

history = {"train_loss": [], "train_acc": [], "test_loss": [], "test_acc": []}

```

```

def run_epoch(loader, train=True):
    model.train() if train else model.eval()
    total_loss, correct, total = 0.0, 0, 0
    context = torch.enable_grad() if train else torch.no_grad()
    with context:
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            if train:
                optimizer.zero_grad()
                outputs = model(images)
                loss = criterion(outputs, labels)
                if train:
                    loss.backward()
                    optimizer.step()
                total_loss += loss.item() * images.size(0)
                correct += (outputs.argmax(1) == labels).sum().item()
                total += labels.size(0)
    return total_loss / total, correct / total

for epoch in range(1, N_EPOCHS + 1):
    train_loss, train_acc = run_epoch(train_loader, train=True)
    test_loss, test_acc = run_epoch(test_loader, train=False)
    history["train_loss"].append(train_loss)
    history["train_acc"].append(train_acc)
    history["test_loss"].append(test_loss)
    history["test_acc"].append(test_acc)
    print(f"Эпоха {epoch:2d}/{N_EPOCHS} | train loss {train_loss:.4f} acc {train_acc*100:5.2f}%
| "
          f"test loss {test_loss:.4f} acc {test_acc*100:5.2f}%")

epochs_range = range(1, N_EPOCHS + 1)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(epochs_range, history["train_loss"], label="Train loss", marker='o', markersize=3)
axes[0].plot(epochs_range, history["test_loss"], label="Test loss", marker='o', markersize=3)
axes[0].set_xlabel("Эпоха"); axes[0].set_ylabel("Loss")
axes[0].set_title("Изменение ошибки при обучении"); axes[0].legend(); axes[0].grid(alpha=0.3)

axes[1].plot(epochs_range, [a*100 for a in history["train_acc"]], label="Train acc", marker='o',
markersize=3)
axes[1].plot(epochs_range, [a*100 for a in history["test_acc"]], label="Test acc", marker='o',
markersize=3)
axes[1].set_xlabel("Эпоха"); axes[1].set_ylabel("Accuracy, %")
axes[1].set_title("Изменение точности при обучении"); axes[1].legend(); axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print(f"Итоговая точность на тесте: {history['test_acc'][-1]*100:.2f}%")

model.eval()
class_correct, class_total = defaultdict(int), defaultdict(int)

with torch.no_grad():
    for images, labels in test_loader:
        images, labels = images.to(device), labels.to(device)
        preds = model(images).argmax(1)
        for label, pred in zip(labels, preds):
            class_total[label.item()] += 1
            class_correct[label.item()] += int(label.item() == pred.item())

```

```

print("Точность по классам:")
for i, cls in enumerate(CLASSES):
    print(f" {cls:>10s}: {100*class_correct[i]/class_total[i]:5.2f}%")

def visualize_test_sample(index=None):
    model.eval()
    if index is None:
        index = random.randrange(len(test_set))
    image, true_label = test_set[index]
    input_tensor = image.unsqueeze(0).to(device)
    with torch.no_grad():
        probs = F.softmax(model(input_tensor), dim=1).cpu().numpy().flatten()
    pred_label = int(np.argmax(probs))

    def denorm(img):
        img = img.numpy().transpose(1, 2, 0) * np.array(STD) + np.array(MEAN)
        return np.clip(img, 0, 1)

    fig, axes = plt.subplots(1, 2, figsize=(10, 4), gridspec_kw={'width_ratios': [1, 1.4]})
    axes[0].imshow(denorm(image)); axes[0].axis('off')
    correct = pred_label == true_label
    axes[0].set_title(f"Истина: {CLASSES[true_label]}\nПредсказание: {CLASSES[pred_label]}",
                     color='green' if correct else 'red')
    colors = ['green' if i == true_label else ('red' if i == pred_label else 'steelblue') for i
in range(10)]
    axes[1].barh(CLASSES, probs * 100, color=colors)
    axes[1].set_xlabel("Вероятность, %"); axes[1].set_xlim(0, 100); axes[1].invert_yaxis()
    plt.tight_layout(); plt.show()

for _ in range(3):
    visualize_test_sample()

def predict_custom_image(path_or_pil):
    model.eval()
    img = Image.open(path_or_pil).convert('RGB') if isinstance(path_or_pil, str) else
path_or_pil.convert('RGB')
    preprocess = transforms.Compose([
        transforms.Resize((32, 32)),
        transforms.ToTensor(),
        transforms.Normalize(MEAN, STD),
    ])
    input_tensor = preprocess(img).unsqueeze(0).to(device)
    with torch.no_grad():
        probs = F.softmax(model(input_tensor), dim=1).cpu().numpy().flatten()
    pred_label = int(np.argmax(probs))

    fig, axes = plt.subplots(1, 2, figsize=(10, 4), gridspec_kw={'width_ratios': [1, 1.4]})
    axes[0].imshow(img); axes[0].axis('off')
    axes[0].set_title(f"Предсказание сети: {CLASSES[pred_label]}")
    colors = ['red' if i == pred_label else 'steelblue' for i in range(10)]
    axes[1].barh(CLASSES, probs * 100, color=colors)
    axes[1].set_xlabel("Вероятность, %"); axes[1].set_xlim(0, 100); axes[1].invert_yaxis()
    plt.tight_layout(); plt.show()
    return pred_label, probs

predict_custom_image(r'C:\Users\User\Desktop\News\doggie.jpg')

```

Результат тестирования:

Используемое устройство: cuda

100%|██████████| 170M/170M [27:00<00:00, 105kB/s]

Train: 50000, Test: 10000, размер изображения: torch.Size([3, 32, 32])

```
SimpleCNN(  
  (conv1): Conv2d(3, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (conv3): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))  
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,  
    ceil_mode=False)  
  (fc1): Linear(in_features=2048, out_features=256, bias=True)  
  (fc2): Linear(in_features=256, out_features=10, bias=True)  
  (dropout): Dropout(p=0.4, inplace=False)  
)
```

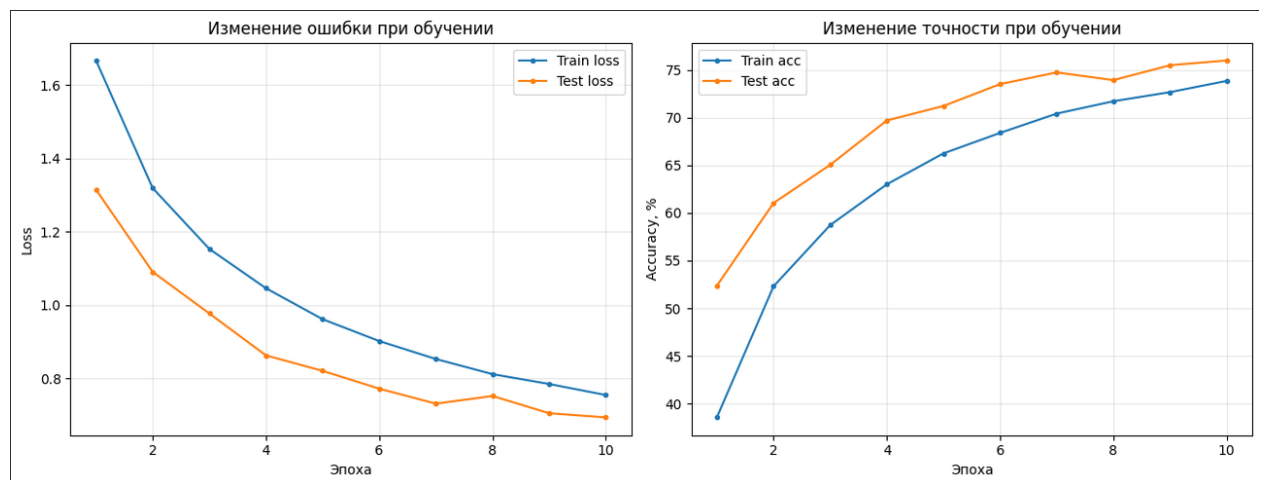
Обучаемых параметров: 620,362

Эпоха	1/10		train loss	1.6672	acc	38.57%		test loss	1.3149	acc	52.37%
Эпоха	2/10		train loss	1.3191	acc	52.30%		test loss	1.0906	acc	61.03%
Эпоха	3/10		train loss	1.1532	acc	58.72%		test loss	0.9772	acc	65.04%
Эпоха	4/10		train loss	1.0459	acc	63.00%		test loss	0.8627	acc	69.69%
Эпоха	5/10		train loss	0.9614	acc	66.24%		test loss	0.8206	acc	71.21%
Эпоха	6/10		train loss	0.9018	acc	68.39%		test loss	0.7720	acc	73.50%
Эпоха	7/10		train loss	0.8528	acc	70.41%		test loss	0.7311	acc	74.73%
Эпоха	8/10		train loss	0.8116	acc	71.71%		test loss	0.7519	acc	73.92%
Эпоха	9/10		train loss	0.7847	acc	72.66%		test loss	0.7050	acc	75.48%
Эпоха	10/10		train loss	0.7546	acc	73.84%		test loss	0.6936	acc	75.98%

Точность по классам:

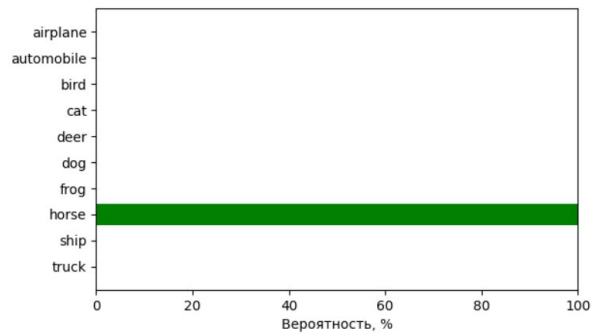
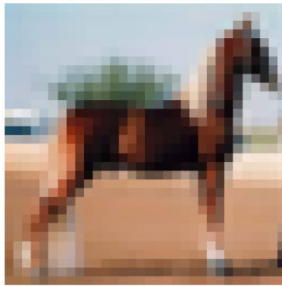
- airplane: 88.60%
- automobile: 93.40%
- bird: 61.10%
- cat: 51.40%
- deer: 74.60%
- dog: 60.60%
- frog: 84.70%
- horse: 79.50%
- ship: 85.40%
- truck: 80.50%

График изменения ошибки:

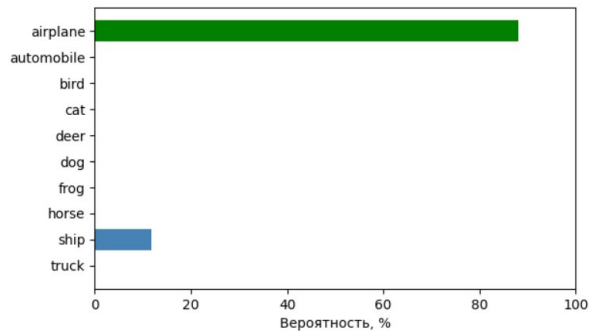


Итоговая точность на тесте: 75.98%

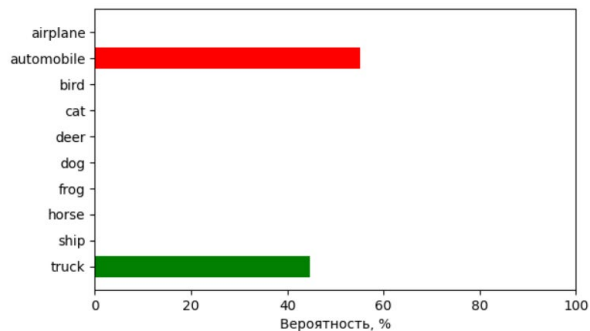
Истина: horse
Предсказание: horse



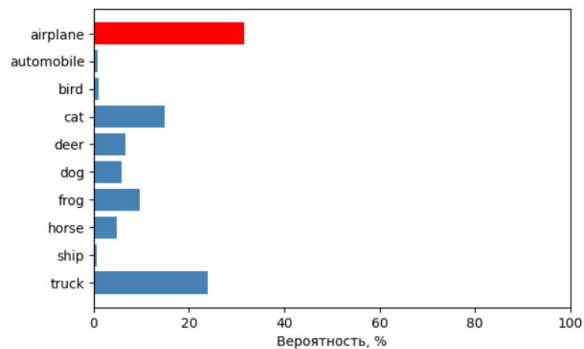
Истина: airplane
Предсказание: airplane



Истина: truck
Предсказание: automobile



Предсказание сети: airplane



Простая свёрточная сеть из базовых слоёв реально способна научиться отличать объекты на картинках – и делает это неплохо, но у неё есть чёткие границы возможностей: она путает похожие друг на друга классы и плохо справляется с картинками, непохожими по стилю на то, на чём её учили. Чтобы приблизиться к результатам топовых современных сетей, нужна не магия, а конкретные усложнения – более глубокая архитектура, специальные приёмы против затухания сигнала в сети, более разнообразные и объёмные данные для обучения.

Вывод: Прodelав данную лабораторную работу я научился конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения